

Prezentacja

code-review, wstępny projekt protokołu i serializacja
sieciowa





Code-review, walidacja otrzymanego projektu i Walidacja kodu (2 błędy znalezione)

W pliku implementacji UAR:

- Zmiana z $e_ = w_ - y_ \rightarrow e_ = _y - w_$

Błąd okresu sinusoidy:

- Ustawienia okresu w ui nie korelowały z symulacją



2 najsilniejsze strony

1. Przejrzystość kodu

```
11
12 struct ProbkaUAR
13 {
14     double t;
15
16     double w;
17     double y;
18     double e;
19     double u;
20
21     double uP;
22     double uI;
23     double uD;
24 };
25
```

```
6 // Tryb całkowania
7 enum class LiczCalk
8 {
9     Zewnetrzne,
10    Wewnetrzne
11 };
```

2. Niekonwencjonalny wygląd GUI (wyświetlacze pomiarowe)





Słabe strony

Powtórzenia i niespójności w kodzie bazowym (DRY)

```
3 RegulatorPID::RegulatorPID(double k)
4     : kP_(k)
5     , Ti_(0.0)
6     , Td_(0.0)
7     , uP_(0.0)
8     , uI_(0.0)
9     , uD_(0.0)
10    , ePrev_(0.0)
11    , sumE_(0.0)
12    , trybCalk_(LiczCalk::Zewnetrzne)
13 {
14 }
15
16 RegulatorPID::RegulatorPID(double k, double Ti)
17     : kP_(k)
18     , Ti_(Ti)
19     , Td_(0.0)
20     , uP_(0.0)
21     , uI_(0.0)
22     , uD_(0.0)
23     , ePrev_(0.0)
24     , sumE_(0.0)
25     , trybCalk_(LiczCalk::Zewnetrzne)
26 {
27 }
28
29 RegulatorPID::RegulatorPID(double k, double Ti, double Td)
30     : kP_(k)
31     , Ti_(Ti)
32     , Td_(Td)
33     , uP_(0.0)
34     , uI_(0.0)
35     , uD_(0.0)
36     , ePrev_(0.0)
37     , sumE_(0.0)
38     , trybCalk_(LiczCalk::Zewnetrzne)
39 {
40 }
```

```
private:
    // Budowa UI / wykresow
    void budujInterfejs();
    void konfigurujWykresy();

    // Synchronizacja / odswiezanie
    void odswiezGUIzKonfiguracji();
    void aktualizujWyswietlaczeNumeryczne(const ProbkaUAR& p);
    void wyczyszcWykresy();
    void onProbka(const ProbkaUAR& p);
    void wyzerujWyswietlacze();

private:
    // Backend / sterowanie
    Ui::MainWindow* ui_ = nullptr;
    UslugiUAR uslugi_;

    int oknoObserwacjiS_ = 10;
    int intervalMs_ = 200; // cache TTms do ost X (margines max 3 probkt)

    // Wykresy
    OCustomPlot* plotWy_ = nullptr;
```

niepotrzebne definiowanie PI, które jest zdefiniowane w cmath

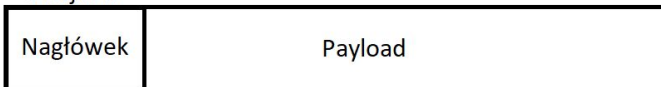
```
2 #include <cmath>
3
4 namespace
5 {
6     constexpr double PI = 3.1415926535897932384
7 }
```

Wstępny projekt protokołu sieciowego (klasy komunikacji sieciowej)

```
struct OdebranaWiadomosc
{
    TypWiadomosci typ = TypWiadomosci::Konfiguracja;
    PayloadWiadomosci payload = KonfiguracjaUAR{};
};
```

```
enum TypWiadomosci : uint8_t {
    Konfiguracja = 1,
    Sterowanie = 2,
    ProbkaSymulacji = 3
};
```

5 bajtów



1 bajt - TypWiadomosci

4 bajty - Rozmiar danych (payloadu)

```
// wyłączenie wyrównywania pamięci na potrzeby binarnych ramek sieciowych
#pragma pack(push, 1)
struct Naglowek {
    TypWiadomosci typ;
    uint32_t rozmiarDanych;
};
#pragma pack(pop)

#pragma pack(push, 1)
struct PakietProbkiSymulacji
{
    uint64_t krok = 0;
    double t = 0.0;
    double w = 0.0;
    double y = 0.0;
};

struct PakietSterowania
{
    uint64_t krok = 0;
    double u = 0.0;
    double uP = 0.0;
    double uI = 0.0;
    double uD = 0.0;
};
#pragma pack(pop)

using PayloadWiadomosci = std::variant<KonfiguracjaUAR, PakietProbkiSymulacji, PakietSterowania>;
```



Jak działa serializacja?

```
std::vector<uint8_t> ProtokolUAR::zbudujRamkeKonfiguracji(const KonfiguracjaUAR &cfg)
{
    return zbudujRamkeWewnetrzna(TypWiadomosci::Konfiguracja, cfg.serializujDoJSON());
}
```

```
std::vector<uint8_t> ProtokolUAR::zbudujRamkeProbkiSymulacji(const PakietProbkiSymulacji &probka)
{
    std::string payload(sizeof(PakietProbkiSymulacji), '\0');
    std::memcpy(payload.data(), &probka, sizeof(PakietProbkiSymulacji));

    return zbudujRamkeWewnetrzna(TypWiadomosci::ProbkaSymulacji, payload);
}
```

```
std::vector<uint8_t> ProtokolUAR::zbudujRamkeSterowania(const PakietSterowania &sterowanie)
{
    std::string payload(sizeof(PakietSterowania), '\0');
    std::memcpy(payload.data(), &sterowanie, sizeof(PakietSterowania));

    return zbudujRamkeWewnetrzna(TypWiadomosci::Sterowanie, payload);
}
```

```
std::vector<uint8_t> ProtokolUAR::zbudujRamkeWewnetrzna(TypWiadomosci typ, const std::string &payload)
{
    Naglowek naglowek;
    naglowek.typ = typ;
    naglowek.rozmiarDanych = static_cast<uint32_t>(payload.size());

    std::vector<uint8_t> ramka;
    ramka.reserve(sizeof(Naglowek) + payload.size());

    const uint8_t *ptr = reinterpret_cast<const uint8_t *>(&naglowek);
    ramka.insert(ramka.end(), ptr, ptr + sizeof(Naglowek));
    ramka.insert(ramka.end(), payload.begin(), payload.end());

    return ramka;
}
```



Jak działa deserializacja?

```
bool ProtokolUAR::dekodujRamke(const std::vector<uint8_t> &ramka,
                                OdebranaWiadomosc &wiadomosc,
                                std::string *blad)
{
    TypWiadomosci typ;
    if (!odczytajTypWiadomosci(ramka, typ, blad))
        return false;

    wiadomosc.typ = typ;

    switch (typ)
    {
        case TypWiadomosci::ProbkaSymulacji:
        {
            PakietProbkiSymulacji p;
            if (!odczytajRamkeProbkiSymulacji(ramka, p, blad))
                return false;
            wiadomosc.payload = p;
            return true;
        }
        case TypWiadomosci::Sterowanie:
        {
            PakietSterowania s;
```

```
std::cout << "\n[ODBIORCA] Wklej kod HEX wygenerowany na nadawcy: ";
std::string hexInput;
std::cin >> hexInput;

std::vector<uint8_t> ramka = zHex(hexInput);

OdebranaWiadomosc wiadomosc;
std::string blad;
if (!dekodujRamke(ramka, wiadomosc, &blad))
{
    std::cout << "Blad: " << blad << "\n";
    return;
}

switch (wiadomosc.typ)
{
    case TypWiadomosci::Konfiguracja:
    {
        const KonfiguracjaUAR &cfg = std::get<KonfiguracjaUAR>(wiadomosc.payload);
```



Połączenie sieciowe

```
if (!serwer.listen(QHostAddress::Any, static_cast<quint16>(port)))  
{
```

```
    klient.connectToHost(QString::fromStdString(host), static_cast<quint16>(port));  
    if (!klient.waitForConnected(10000))  
    {  
        std::cout << "Nie mozna polaczyc: " << klient.errorString().toStdString() << "\n";  
        return;  
    }  
}
```

```
socket = &klient;  
std::cout << "[KLIENT] Polaczono z serwerem.\n";
```

```
static bool wyslijRamke(QTcpSocket* socket, const std::vector<uint8_t>& ramka)  
{  
    if (!socket || ramka.empty())  
        return false;  
  
    const char* data = reinterpret_cast<const char*>(ramka.data());  
    const quint64 expected = static_cast<quint64>(ramka.size());  
    const quint64 sent = socket->write(data, expected);  
    if (sent != expected)  
        return false;  
  
    return socket->waitForBytesWritten(3000);  
}
```


Zmiany w GUI oraz nowe okno

Konfiguracja połączenia sieciowego

Konfiguracja Instancji

Typ instancji: Regulator (taktuje)

Tryb sieciowy: Klient (łączy do serwera)

Parametry Połączenia

Adres IP: 127.0.0.1

Port TCP: 5050

Ostrzeżenie: Przejście w tryb sieciowy zablokuje część lokalnych kontrolek!!!

Rozpocznij Połączenie Anuluj

SYMULACJA

START STOP

RESET

Interwał TTms: 200 ms

Okno obserwacji: 10 s

POMIARY

w: 0 y: 0

e: 0 u: 0

uP: 0 ul: 0

uD: 0

SIEĆ

Ustawienia Sieciowe

Tryb: Stacjonarny

● Symulacja wyrabia

MODEL ARX

Parametry ARX

PARAMETRY PID

Wzmocnienie (Kp): 0,50

Całkowanie (Ti): 5,00

Różniczkowanie (Td): 0,20

Tryb całkowania: Zewnętrzne

Reset I Reset D

GENERATOR WARTOŚCI ZADANEJ

Typ sygnału: Prostokąt